# Automation QE Platform: Case Study and Challenge

AscendQE - Automation QE Platform in action: one delivery example and one problem solved

**Illustrative samples.** The client, figures and timelines below are sample data created to show how the platform works. Replace them with real client results before using this document outside AscendQE. The specialist-to-step mapping is also illustrative, based on each specialist's function; the official mapping is not yet defined.

## 1. Case study: regression automation for a retail banking portal

### The client

A mid-size bank (sample client, "Northfield Bank") ran its customer portal regression entirely by hand: **620 test cases**, tested by 5 testers over **3 days** for every fortnightly release. Release windows kept slipping and testers had no time for exploratory work.

### Setup by the Project Admin

- Created a project on the Automation QE Platform and uploaded the baselined requirements and business rules (48 documents) into the project RAG store.
- Connected the Git repository and the Jenkins pipeline; stored all credentials as Vault references.
- Set the framework and language, a 3-browser matrix (Chrome, Edge, Firefox), 8 parallel workers and a retry policy of 1 rerun.
- Passed readiness validation, then activated the flow and added 4 Project Members and 2 Viewers.

### How the agent handled it

A Project Member submitted one intent: *"Automate the login, fund transfer and statement flows."* myHelm checked the role and project scope, created a mission with an audit ID and routed it through the fixed flow.

| # | Specialist | What it did | Sample output |
|---|---|---|---|
| 1 | Sherpa (Brief) | Turned the intent into a scoped brief tied to the uploaded requirements | 3 flows, 62 requirements in scope |
| 2 | Scout (Discover) | Crawled the portal and its APIs | 84 screens, 1,240 fields, 96 API endpoints |
| 3 | Sage (Design) | Proposed positive, negative and boundary scenarios | 540 scenarios incl. transfer-limit and invalid-amount cases |
| 4 | Trace (Prepare) | Generated valid, invalid, boundary and duplicate data and kept it for reuse | 1,850 data sets |
| 5 | Bit (Automate) | Wrote UI and API automation using the approved repository patterns | 540 automated tests, 3 pull requests |
| 6 | Sentry (Review) | Put code and data in front of human reviewers before merge | 21 review comments, all resolved |
| 7 | Dash (Run) | Executed suites across the browser matrix and collected evidence | Logs, screenshots, videos and traces per test |
| 8 | Atlas (Manage) | Kept missions, results and audit together for the team | One dashboard for all runs |

### Timeline (sample)

| Week | Activity |
|---|---|
| 1 | Project setup, document upload, integrations, readiness validation |
| 2 | Discovery and test design; reviewer sign-off on scenarios |
| 3 to 4 | Test data and automation generation; human code review and merge |
| 5 | First full run across 3 browsers; defect triage with the client |

### Outcome (sample figures)

| | | | |
|---|---|---|---|
| **87%** of regression cases automated (540 of 620) | **4.5 hrs** regression cycle per release (was 3 days) | **23** defects found in first run (4 high) | **108 hrs** manual effort saved per release |

| Measure | Before | After |
|---|---|---|
| Regression cycle | 3 days, 5 testers (about 120 person-hours) | 4.5 hours unattended plus about 12 person-hours of review |
| Cases still manual | 620 | 80 (exploratory and visual checks) |
| Defects escaping to production | Baseline | 38% fewer over the next 3 releases |
| Release cadence | Slipped about 1 in 3 releases | No slips in 3 releases |

## 2. Automation challenge: tests break after a UI framework upgrade

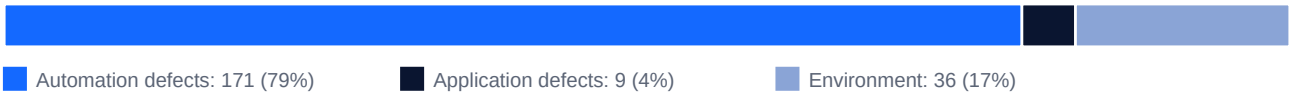
### The challenge

Two weeks after go-live, the bank upgraded its front-end framework. The next nightly run failed **216 of 540 tests (40%)**. On the dashboard every failure looked the same, so the team could not tell real defects from broken locators or a slow environment. Manual triage was estimated at **2.5 days**.

### Why it was hard

- Three different causes were mixed together: changed locators, genuine application bugs and environment timeouts.
- Re-running everything would have wasted a night; healing everything blindly could have hidden real defects.

**Classification of the 216 failed tests (sample)**




### How it was overcome

| # | Specialist | Action | Sample result |
|---|---|---|---|
| 1 | Dash (Run) | Evidence was already captured for every failure | 216 failures with screenshots, traces and logs |
| 2 | Pip (Investigate) | Analysed the evidence and classified each failure | 171 automation, 9 application, 36 environment |
| 3 | Patch (Repair) | Repaired only the automation defects by updating locators, then re-ran them | 158 of 171 healed (92%); all 158 passed on re-run; 13 flagged for manual rewrite |
| 4 | Sentry (Review) | Sent every proposed fix to a human reviewer before it was applied | 158 fixes approved by 2 reviewers in about 40 minutes |

| # | Specialist | Action | Sample result |
|---|---|---|---|
| 5 | Dash (Run) | Re-ran environment failures under the approved retry policy | 33 of 36 passed; 3 escalated to the environment team |
| 6 | Yeti (Improve) | Recommended stable locator strategies (test IDs, role-based selectors) | 410 selectors moved to test IDs |
| 7 | Echo (Collaborate) | Shared the triage summary and raised the real defects with developers | 9 application defects logged with evidence |

## Result (sample figures)

| | | | |
|---|---|---|---|
| **92%** of automation failures repaired automatically | **9** real defects raised, none masked | **3 hrs** triage time (manual estimate: 2.5 days) | **6%** failures after the next UI update (was 40%) |

**Lesson.** The platform repaired only failures that were automation defects, and a human approved each fix before it landed. That kept self-healing from hiding genuine product bugs, and the locator improvements made the suite far more resilient to the next UI change.